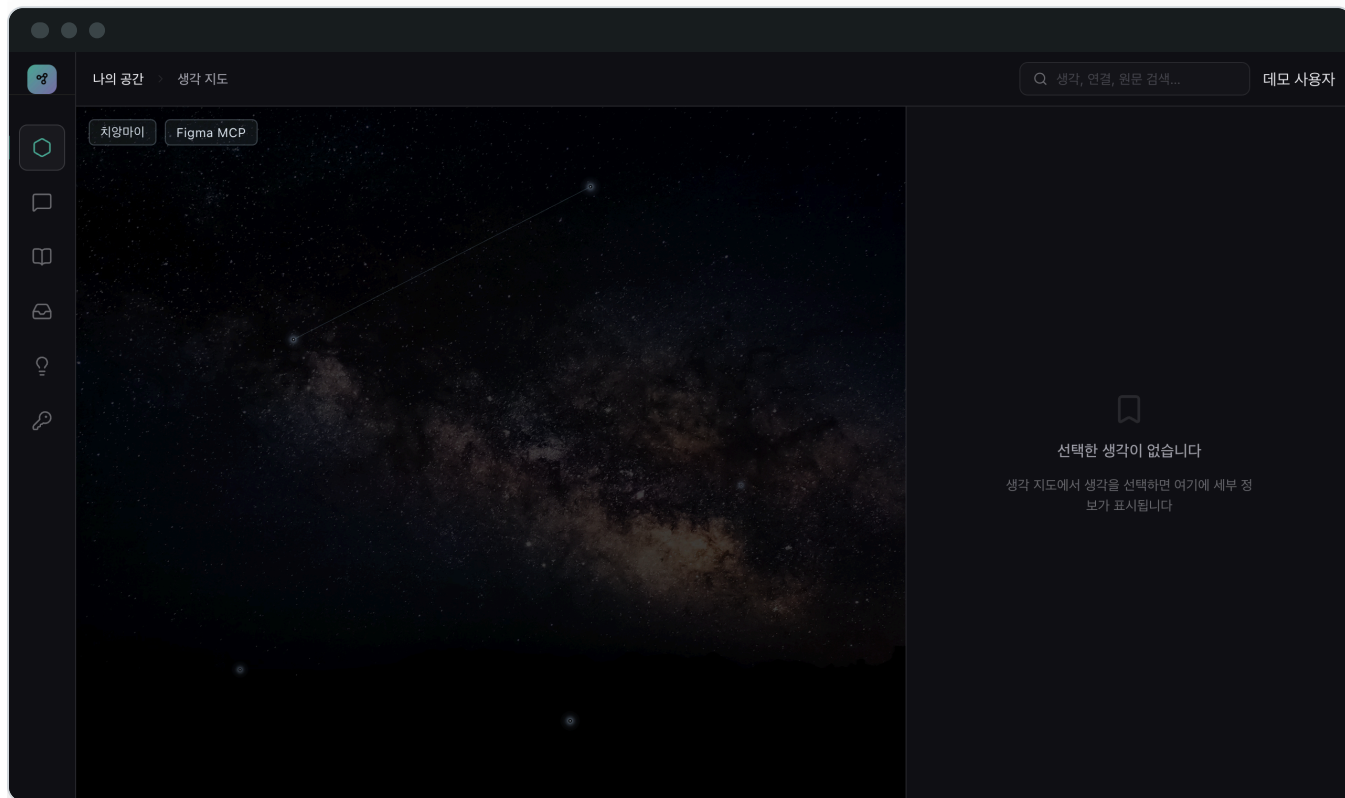


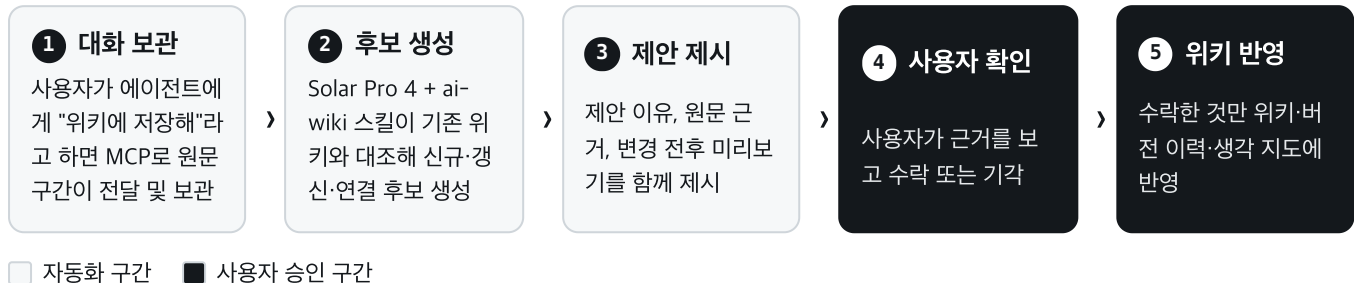
AI 대화를 승인 통제형 개인 위키와 생각 지도로 바꾸는 서비스



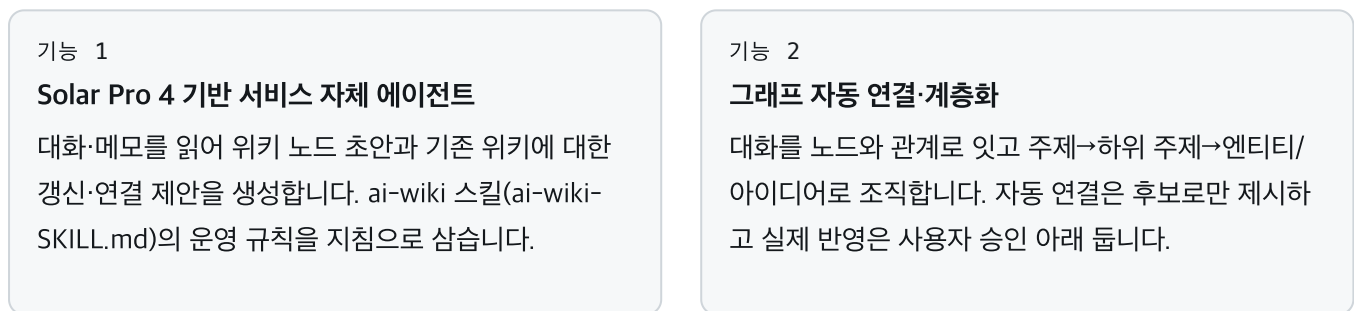
00 한눈에 보는 Nodus

보관 → 후보 생성 → 제안 → 사용자 확인 → 반영 파이프라인

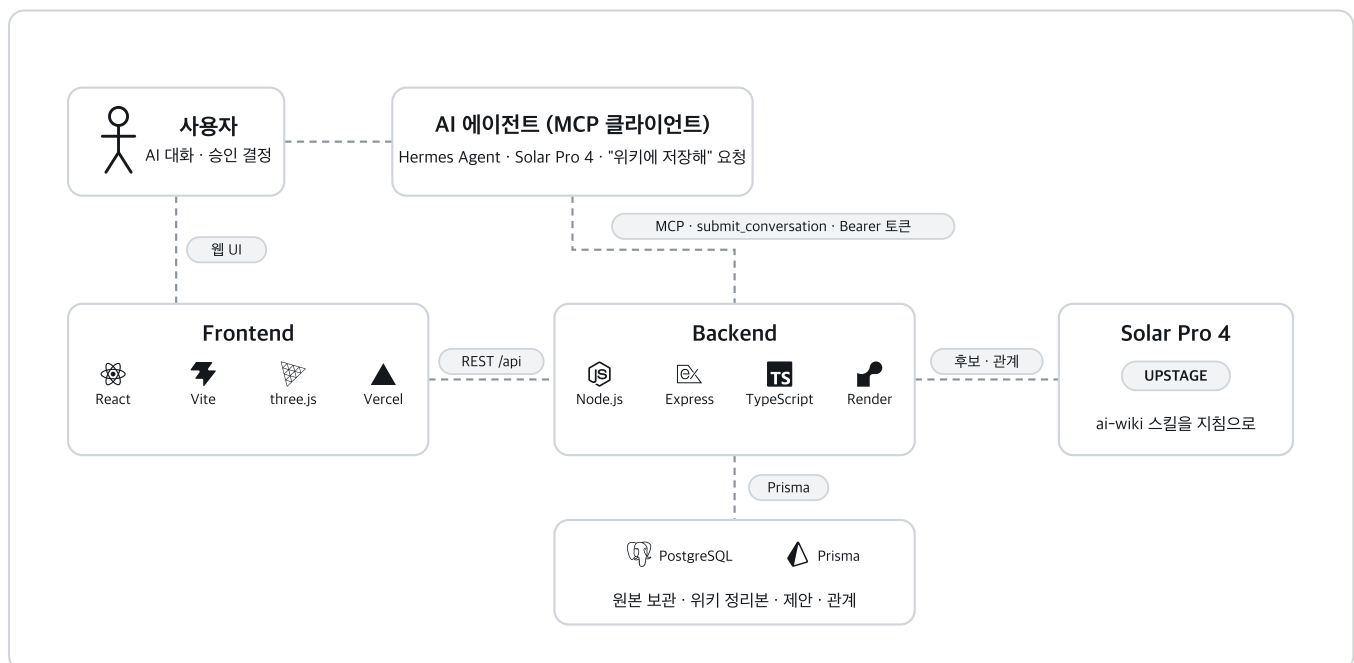
승인을 받지 않은 내용은 위키에 쓰이지 않습니다.



주요 기능



아키텍처



1.1 제품 설명

사용자가 AI와 주고받은 대화를 읽고, **개인화된 주제별 위키**로 정리해 제공하는 **웹 서비스**입니다.

에이전트가 생성한 결과는 위키에 바로 쓰지 않고 **제안 형태로만 제시**하며, 사용자가 **수락**한 내용만 실제 개인 위키에 반영됩니다. 즉, 자동 정리 및 그래프 구축과 사람의 통제(수락/기각)를 함께 제공합니다.

개발자뿐 아니라 **AI를 사용하는 모든 사용자**를 대상으로 하며, 기술 지식뿐 아니라 아이디어·결정·계획·배움·회고 등 일상적 대화도 위키 형태로 다룰 수 있어야 합니다.

자동과 승인의 구분

여기서 말하는 자동 제안 및 자동 정리란 대화를 읽어 위키 노드·제안 후보를 만드는 구간까지를 뜻합니다. 후보를 실제 위키에 반영하는 단계는 반드시 사용자 승인을 거칩니다. 즉 **후보 생성은 자동화 영역, 실제 반영은 사용자 승인 영역**입니다.

1.2 목표

- ✓ 개발자뿐 아니라 AI를 사용하는 일반 사용자도 부담 없이 쓸 수 있는 낮은 진입장벽과 친근한 경험을 제공합니다.
- ✓ 외부 AI 대화를 MCP를 통해 사용자별로 안전하게 수신하고 보관합니다.
- ✓ Solar Pro 4가 ai-wiki 스킴으로 대화를 읽어 신규 위키 생성, 기존 위키 갱신, 위키 간 연결을 제안합니다.
- ✓ 사용자가 변경 내용과 원문 근거를 확인한 뒤 제안을 수락하거나 기각할 수 있도록 합니다.
- ✓ 수락된 결과만 개인 위키와 생각 지도에 반영합니다.
- ✓ 같은 주제의 후속 대화가 기존 위키의 갱신 제안으로 이어지도록 합니다.
- ✓ Solar가 AutoSchema의 스키마 유도 방식을 참고해 노드의 개념적 역할과 관계 유형을 판단하도록 합니다.

1.3 범위

포함	제외
AI와 주고받은 대화를 주제별 위키로 정리	단순 채팅 로그 보관만 하고 구조화하지 않는 방식
새 대화 유입 시 기존 위키 대조 후 신규·갱신·연결 제안 (초반 보수적 제안 + 근거 제시)	근거 없이 연관성만 나열하는 추천 스팸성 기능
과거 아이디어와 최근 맥락 사이 유용한 관계를 근거와 함께 제시	근거 표기 없는 자동 연관성 엔진
개발자뿐 아니라 일반 사용자도 쓸 수 있는 친근한 인터페이스와 낮은 진입장벽	특정 개발 도구/언어 전용 지식베이스로 한정
MCP를 통한 사용자 요청 기반 대화 원문 수신·보관	사용자 요청 없이 대화를 자동 수집하는 기능

포함	제외
Solar Pro 4가 ai-wiki 스킬로 만드는 신규·갱신·연결 제안	AI가 생성한 내용을 사용자 승인 없이 바로 위키에 반영하는 기능
변경 전후와 원문 근거를 확인한 뒤 수락·기각	제안 내용을 사용자가 직접 편집하는 기능
AutoSchema의 스키마 유도 방식을 참고한 관계 유형 추론	AutoSchemaKG 전체 파이프라인 및 자동 스키마 데이터베이스 구축
사용자가 승인한 위키 관계를 생각 지도에 표시	승인되지 않은 관계를 자동 확정하는 기능
위키와 보관 대화에 대한 기본 키워드 검색	개인화 랭킹·벡터 검색·그래프 경로 검색
TBD: 추후 사용자 유형별 확장(개인 지식 정리, 팀 공유 등)	지금 단계에서 확정할 세부 사용자 유형 분류

02 문제 정의

2.1 배경

- AI를 자주 쓰는 사람들은 대화 내용이 쌓이지만, 나중에 다시 찾아보기 어렵습니다.
- 기록 자체가 흩어져 있으면, 예전 아이디어가 지금 맥락과 연결되는 지점을 놓치기 쉽습니다.
- 위키로 정리하면 꺼내 쓰기 좋지만, 사람이 매번 정리하기엔 부담이 큼니다.

2.2 해결할 문제

문제 1

기록으로만 남는 대화

AI와 주고받은 대화가 기록으로만 남고, 주제별 지식으로 다시 꺼내 쓰기 어렵습니다.

문제 2

연결되지 않는 새 대화

새 대화가 들어와도 기존 기록과 연결되지 않아, 과거 아이디어와의 관계가 표면화되지 않습니다.

문제 3

높은 진입장벽

개발자 중심으로만 설계된 도구는 비개발자가 쓰기엔 진입장벽이 높습니다.

2.3 기대 효과

- 사용자가 따로 정리하지 않아도 대화 기록이 위키 형태로 남아 재탐색 비용이 낮아집니다.
- 새 대화 유입 시 기존 위키와의 관계가 제안되어, 위키가 정적 저장소가 아니라 진화하는 지식 기반이 됩니다.
- 과거와 현재 맥락 사이의 관계가 근거와 함께 제시되어, 예전 아이디어를 지금 상황에서 다시 활용할 가능성이 높아집니다.
- 비개발자도 쓸 수 있는 경험을 제공함으로써, AI를 사용하는 더 넓은 사용자층에게 가치를 전달할 수 있습니다.
- 대화가 일회성 채팅으로 끝나지 않고 신규 위키 또는 기존 위키의 갱신안으로 누적됩니다.
- 사용자는 제안의 변경 내용과 원문 근거를 확인한 후 수락 여부를 결정할 수 있습니다.
- 기존 위키 사이의 관계가 근거와 함께 제안되어, 따로 보관된 생각을 연결해서 볼 수 있습니다.
- 승인된 관계가 생각 지도에 표시되어 사용자가 자신의 지식 구조를 시각적으로 탐색할 수 있습니다.

03 목표 사용자

3.1 주요 사용자

사용자 1 AI로 아이디어를 정리하며 일하는 개인	블로그를 쓰거나 공부하거나 기획하는 비개발자. AI와 자주 대화하지만 기록이 흩어져 있어, 예전에 했던 대화를 이어가려면 매번 찾고 다시 설명하는 데 힘이 드는 상황 니즈 대화에서 자동으로 주제별 정리, 예전 생각과 지금 생각 사이 연결 페인 기록할 시간은 부족하고, 쌓아둔 대화는 다시 안 봄
사용자 2 여러 주제를 AI와 대화하며 배우는 사용자	학습, 취미, 의사결정, 회고 등 다양한 맥락에서 AI와 소통하는 사람. 주제가 여러 갈래로 늘어나면 어느 대화에서 무엇을 배웠는지 되짚기 어려워, 같은 질문을 다시 하거나 처음부터 설명하는 데 힘이 드는 상황 니즈 기술 중심이 아니라 아이디어·결정·배움 단위로 위키가 생기길 원함 페인 대화 내용이 많아져도 구조가 없어서 전체 그림이 안 보임
사용자 3 · 향후 소규모 팀 / 커뮤니티	여러 사람이 AI와 주고받은 맥락을 함께 보고 싶은 팀. 지금은 구체화하지 않고 TBD로 둡니다.

3.2 사용자 여정 요약

터치포인트	현재의 고통 지점	Nodus가 개입하는 방식
대화 발생	기록이 흩어지고 끝나버림	"위키에 저장해" 한마디로 MCP를 통해 원문을 보관
위키 확인	별도 정리 없으면 찾기 어려움	Solar가 주제별 노드 초안을 만들고 사용자가 수락해 축적
새 대화 유입	기존 위키와 대조해 연결이 보이지 않음	기존 위키와 대조해 갱신·연결 제안을 근거와 함께 제시
과거 회상	예전 아이디어가 지금 맥락과 다시 연결되어도 표면화되지 않음	승인된 관계를 생각 지도에 표시하고 키워드로 위키·보관 대화를 검색

이 제품은 여정의 각 지점에서 **정리·제안·관계 제시**를 통해 꺼내는 비용을 낮추는 것을 목표로 합니다.

04 기능 요구사항

4.1 핵심 기능 (MVP 기준)

ID	기능	우선순위
F1	대화 읽어서 위키 노드 만들기	P0
F2	제안 수락/기각	P0
F3	새 대화 유입 시 기존 위키 대조 후 제안(신규·갱신·연결)	P0
F4	위키와 보관 대화 검색	P1
F5	Solar 기반 동적 관계 스키마 유도 및 연결 제안	P0

F1: 대화 읽어서 위키 노드 만들기

P0

MCP로 전달된 대화 원문을 Solar Pro 4가 ai-wiki 스킴으로 분석해 주제별 위키 노드 후보를 만듭니다. 기존 위키와 같은 주제라면 신규 노드보다 기존 위키 갱신을 우선합니다.

"AI와 나는 대화 기록을 선택하면, 주제별로 정리된 위키 노드가 자동으로 만들어지면 좋겠다."

입력

- MCP를 통해 전달된 대화 원문
- 기존 위키 노드

출력 / 결과

- 제목, 요약, 본문, 주제, 태그, 분류를 포함한 신규 위키 후보
- 또는 기존 위키 갱신 제안

제약: 생성 결과는 위키에 바로 반영하지 않고 제안 상태로 저장합니다.

F2: 제안 수락/기각

P0

사용자는 자동 생성된 제안의 내용과 근거를 검토한 뒤 수락하거나 기각할 수 있습니다.

출력 / 결과

- 수락된 신규 제안은 위키 노드로 저장
- 수락된 갱신 제안은 기존 위키와 버전 이력에 반영
- 수락된 연결 제안은 위키 관계로 저장
- 기각된 제안은 위키에 반영하지 않음
- 결정 상태와 시각은 새로고침 후에도 유지

제약: 사용자가 제안 내용을 직접 편집하는 기능은 MVP 범위에 포함하지 않습니다.

F3: 새 대화 유입 시 기존 위키 대조 후 제안

P0

새로운 대화가 들어오면 기존 위키를 검토해, 신규 생성·갱신·연결이 필요해 보이는 내용을 사용자에게 제안합니다.

"새로운 대화를 넣으면 기존 위키를 보고 '이 내용을 여기에 붙이면 좋겠다' 같은 제안을 해주면 좋겠다."

입력

- 새로 들어온 대화
- 기존 위키 노드

출력 / 결과

- 신규 위키 생성 제안
- 기존 위키 갱신 제안
- 기존 위키 간 연결 제안
- 제안 이유와 원문 근거
- 갱신 제안의 변경 전후 미리보기

비고 / 제약

- 초반에는 가장 신뢰도 높은 제안만 보수적으로 보여줍니다.
- 제안은 결과만 던지는 게 아니라, **사용자가 수락하기 전에 무엇이 바뀌는지 알 수 있어야 합니다.**
- 사용자가 수락/기각할 수 있어야 하며, 그 결정과 결과는 다음 새로고침/재접속 후에도 남아 있어야 합니다.

F4: 위키와 보관 대화 검색

P1

사용자는 키워드로 승인된 위키와 서비스에 보관된 대화를 함께 검색할 수 있습니다.

"예전에 저장한 위키나 대화 내용을 키워드로 다시 찾고 싶다."

입력

- 사용자의 검색어

출력 / 결과

- 검색어가 포함된 위키 노드
- 검색어가 포함된 보관 대화
- 검색 결과에서 위키 상세 또는 원본 대화로 이동

제약: 개인화 랭킹, 벡터 검색, 그래프 경로 검색은 현재 MVP 범위에 포함하지 않습니다.

F5: Solar 기반 동적 관계 스키마 유도 및 연결 제안

P0

Solar Pro 4는 AutoSchema의 데이터 기반 스키마 유도 방식을 참고해, 기존 위키 노드와 새 대화에서 노드의 개념적 역할과 관계 유형을 추론합니다. 이를 바탕으로 근거가 있는 위키 연결 후보를 생성합니다.

"서로 관련된 위키가 있다면 어떤 관계인지와 왜 연결되는지 확인하고 직접 승인하고 싶다."

입력

- 새 대화 원문
- 기존 위키 노드
- 원문 세그먼트

출력 / 결과

- 출발 위키와 도착 위키
- 각 위키의 개념적 역할
- 관계 유형
- 관계 판단 이유
- 원문 근거

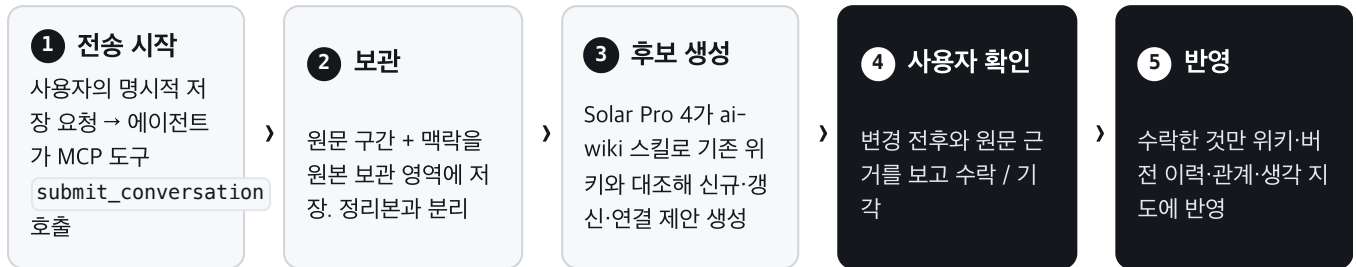
제약

- 공통 단어 또는 태그가 있다는 이유만으로 연결하지 않습니다.
- 관계의 의미와 원문 근거가 명확하지 않으면 제안하지 않습니다.
- 관계 후보는 제안 목록에만 저장하며 사용자 승인 전에는 확정하지 않습니다.
- 사용자가 수락한 관계만 생각 지도에 표시합니다.

구현 범위: AutoSchemaKG 전체 라이브러리를 실행하는 것이 아니라, 데이터에서 개념 유형과 관계 유형을 유도한 뒤 관계를 추출하는 핵심 방식을 Solar의 구조화 출력에 적용합니다.

04-1 데이터 수신 · 저장 · 반영 흐름

MVP에서의 데이터 전달과 위키 반영 흐름을 정의합니다. 자동 제안이 곧바로 반영되는 구조가 아니라 **보관** → **후보 생성** → **제안** → **사용자 확인** → **사용자 선택 반영** 순서로 진행됩니다.



(1) 전송 시작 방식

- MVP에서는 사용자가 명시적으로 "이 내용을 위키에 저장해줘"라고 요청할 때만 전송을 시작합니다. 자동 스트리밍, 주기 전송, 대화 종료 자동 전송은 제외합니다.
- 전송은 Solar 기반 에이전트 → 우리 서비스 MCP 도구 호출 경로로 이뤄지며, 대화 내용(구간 + 맥락)이 전달됩니다.
- 저장 요청의 해석: "위키에 저장해"처럼 말하면 저장 요청으로 해석하고 별도 확인 없이 `submit_conversation` 을 호출합니다.
- 구간 지정 우선: 사용자가 특정 구간을 지정했으면 그 구간만 보냅니다. 지정하지 않았고 현재 주제가 진행 중이면, 위키에 남길 가치가 있는 내용(주제·아이디어·결정·계획·배움·회고)이 포함된 하나의 주제 범위로 묶어 보냅니다.
- 질문은 불명확할 때만: 어떤 구간·주제인지 에이전트도 판단할 수 없을 때만 범위를 확인하는 질문을 합니다.
- 보이는 대화만 전송: 에이전트에게 실제로 보이는 내용만 보내며, 없는 대화를 추론해 만들어내지 않습니다.
- 저장 요청 없으면 전송하지 않음: 대화가 끝났어도 자동으로 전송하지 않습니다.
- 에이전트 지침에서 저장 요청 처리의 연결 위치를 찾을 수 없으면 임의로 행동하지 않고 누락 사실을 보고합니다.

(2) 전송 단위 및 입력 계약

`submit_conversation` 은 역할 구분 메시지 목록(`messages`) 또는 대화 텍스트(`conversation_text`) 중 하나로 원문 구간을 전달합니다. 둘이 함께 오면 불일치 검사를 수행합니다.

필드	필수	설명
<code>session_id</code>	필수	대화/세션을 식별하는 ID
<code>conversation_text</code>	선택	전송할 대화 원문(구간). <code>messages</code> 가 없을 때 사용
<code>messages[]</code>	선택	역할이 구분된 메시지 목록. 각 항목은 <code>role</code> , <code>content</code> 필수, <code>record_id</code> , <code>timestamp</code> (ISO 8601) 선택. 제공되면 원문의 1차 출처
<code>source</code>	선택	전송 출처 구분. 예: <code>mcp</code> , <code>agent</code> , <code>user_direct</code>
<code>context</code>	선택	주제, 스킬, 세션 유형, 앞뒤 흐름 등 보충 맥락. 대화 원문 자체는 넣지 않는다

입력 계약 원칙

- role과 content 중 하나라도 누락되면 도구 오류로 처리합니다. 알 수 없는 id나 시각을 추측해 넣지 않습니다.
- messages와 conversation_text가 함께 오면, 메시지를 역할과 함께 연결한 재구성 텍스트와 conversation_text를 공백·개행 정규화 후 비교합니다. 실질적으로 같으면 정상 처리하고, 다르면 도구 오류로 처리합니다.
- 기존 conversation_text 기반 호출은 계속 지원합니다.

예시 (messages 방식)

```
{
  "session_id": "sess-12345",
  "source": "agent",
  "messages": [
    { "role": "user", "content": "동네 산책 사진으로 사진집을 만들까 고민 중이야.", "record_id": "msg-001", "timestamp": "2026-09-15T10:00:00+09:00" },
    { "role": "assistant", "content": "사진집은 인쇄와 전자책 두 가지 형식이 있어요. 어떤 쪽으로 생각하세요?", "record_id": "msg-002", "timestamp": "2026-09-15T10:00:15+09:00" },
    { "role": "user", "content": "아직 정리는 안 됐고, 일단 아이디어만 보관해두고 싶어.", "record_id": "msg-003", "timestamp": "2026-09-15T10:00:30+09:00" }
  ],
  "context": { "topic": "사진집 기획", "skill": "ai-wiki" }
}
```

(3) 전달 후 보관

- 전달받은 기록은 먼저 **보관**합니다. 보관은 위키 반영과 분리하며, 나중에 정리·제안·검색에 쓸 원본성 기록으로 남깁니다.
- 보관은 두 층입니다. **서비스 수신 원본**(MCP로 전달받은 구간 + 맥락)과 **사용자 추가 원본**(사용자가 직접 덧붙이거나 외부에서 가져온 원본).
- 보관한 기록은 곧바로 위키로 바꾸지 않고, Solar Pro 4가 ai-wiki 스킬로 읽어 위키 생성/변경 후보를 만드는 입력으로 씁니다.

(4) 위키 생성/변경 제안 · (5) 사용자 확인 및 선택 반영

- 전달받은 대화 내용을 바탕으로 Solar Pro 4가 ai-wiki 스킬로 신규·갱신·연결 제안을 만들고, 적용 시 무엇이 바뀌는지 (변경 전/후)를 함께 보여줍니다.
- 사용자는 제안을 확인한 뒤 변경 전/후를 보고 수락 또는 기각합니다. 결정과 결과는 새로고침·재접속 후에도 남습니다.

(6) MVP에서 다루지 않는 것

- 전체 대화를 항상 통째로 보내는 방식, 자동 재전송/주기 동기화, 사용자 요청 없이 대화 종료 시 자동 위키화
- 2차 릴리즈 후보: 위키 노트 타입 확장(아이디어·결정·계획·배움·회고), 사용자 유형별 경험 세분화, 위키 공유/협업, 더 정교한 관계 그래프 시각화, 사용자 피드백을 활용한 제안 품질 지속 개선

05 비기능 요구사항

항목	요구사항
성능	위키 열람과 제안 제시는 사용자가 기다림을 느끼지 않는 수준이어야 함. 대화 파싱/요약/대조는 백그라운드 또는 비동기 처리 검토
보안	사용자 대화/지식 데이터는 민감할 수 있으므로 저장·전송 시 적절한 보호 필요. 사용자별 데이터 격리 필수. MCP 연동은 사용자별 발급 토큰(Bearer)으로 인증
가용성 / SLA	MVP 기준으로 핵심 위키 열람/제안은 안정적으로 제공되어야 함
확장성	위키 노드와 관계가 늘어나도 탐색/제안이 무너지지 않도록 구조 설계 필요
호환성	웹 기반이므로 주요 브라우저에서 정상 동작해야 함
접근성	비개발자 사용자도 쓰므로 읽기/탐색/제안을 이해하기 쉬워야 함
로깅 / 모니터링	제안 수락/기각, 위키 조회 등 핵심 행동 로그를 남겨 품질 개선에 활용
규제 / 컴플라이언스	사용자 데이터 처리 관련 요구사항이 있으면 반영

06 사용자 경험 / UI 요구사항

6.1 주요 화면 / 플로우

화면 1

생각 지도 (홈)

승인된 위키 노드와 관계를 그래프로 탐색합니다. 노드를 선택하면 요약과 정리 본문, 전체 보기, 대화로 이어가기를 제공합니다.

화면 2

내 위키 · 위키 상세

주제별 노드 목록과 개별 노드의 제목·요약·본문·주제 태그·갱신 이력.

화면 3

제안 목록 · 제안 상세

신규·갱신·연결 제안을 유형·상태 배지와 함께 나열하고, 상세에서 변경 전후 비교와 원문 근거를 확인한 뒤 수락 또는 기각합니다.

화면 4

유입 기록 · MCP 연결 · 검색

보관된 대화 원문 목록, 에이전트 연동용 토큰 발급 화면, 위키와 보관 대화를 함께 찾는 검색 결과.

6.2 디자인 원칙

- **낮은 진입장벽:** 개발자뿐 아니라 AI를 쓰는 일반 사용자도 처음 보고 이해할 수 있어야 합니다.
- **명료성:** 신규/갱신/연결 같은 개념도 무겁지 않게, 짧고 직관적으로 표현합니다.
- **통제감:** 자동 정리가 기본이지만 반영 여부는 항상 사용자가 수락/기각으로 결정합니다.
- **자동과 승인의 분리:** 대화 읽기와 후보 생성은 자동화 구간, 실제 위키 반영은 승인 구간입니다. 승인 전에 변경 전/후를 확인하고 결정하게 합니다.
- **근거 투명성:** 관계/제안을 제시할 때 왜 그런지 근거를 함께 보여줍니다.
- **과도한 자동화 회피:** 초반에는 신뢰 높은 제안만 보수적으로 보여주고 사용자에게 피로가 쌓이지 않게 합니다.

6.3 에러 / 예외 처리

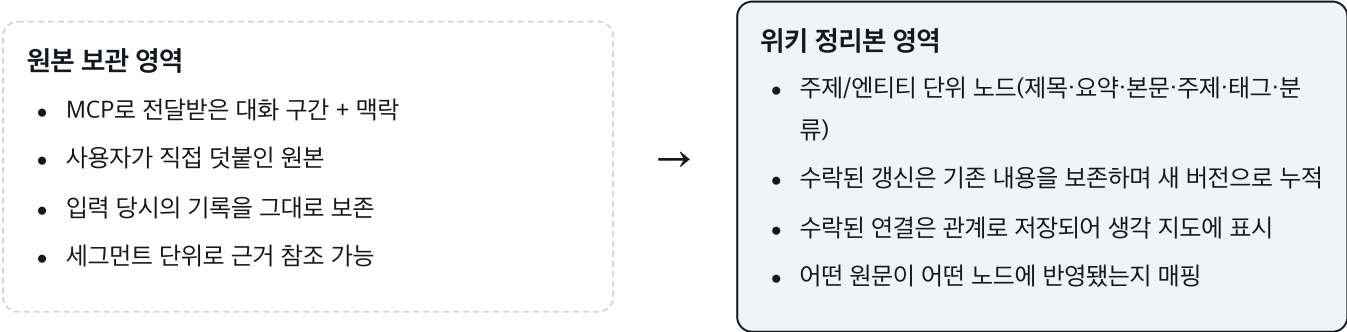
- 대화/스킬 데이터를 읽지 못할 때: 유입 기록에 실패 상태와 사유를 남기고 사용자에게 안내합니다.
- 위키 구조가 아직 비어 있거나 너무 적을 때: 빈 상태 화면에서 다음 행동(대화 저장, 제안 확인)을 안내합니다.
- 제안의 근거가 약해서 표시하지 않는 경우: 제안을 만들지 않거나 "현재 근거 충분치 않음"으로 표기합니다.

7.1 핵심 엔티티

엔티티	역할
사용자 (User)	모든 데이터의 격리 단위. MCP 연결 토큰의 소유자
유입 기록 (Record)	MCP 또는 직접 입력으로 들어온 대화 원문과 맥락. 원본 보관 영역
대화 세그먼트 (ConversationSegment)	원문을 근거 단위로 나눈 조각. 제안의 근거(evidence)가 가리키는 대상
위키 노드 (WikiNode) · 노드 버전 (NodeVersion)	주제/엔티티 단위 정리본과 갱신 이력
제안 (Proposal) · 제안 근거 (ProposalEvidence)	신규·갱신·연결 후보, 변경 전/후, 상태(제안됨·승인됨·기각됨·반영됨), 결정 시각, 원문 인용
노드 관계 (NodeRelationship)	수락된 연결 제안에서 생긴 관계. 관계 유형과 근거를 가지며 생각 지도에 표시
원본→노드 매핑 (RecordToNode)	어떤 원문 구간이 어떤 노드에 반영됐는지 추적
MCP 연결 토큰 (McpCredential)	에이전트가 사용자 계정으로 대화를 제출할 때 쓰는 Bearer 토큰

제안 후보는 "생성됨"과 "승인/기각/반영됨" 상태를 구분해 관리합니다. 자동으로 만든 후보와 실제 위키에 반영된 결과를 혼동하지 않도록 제안→승인→반영 흐름을 별도 상태·이력으로 추적합니다.

7.1.1 위키 저장 구조



- 분리 원칙:** 원본과 정리본이 섞이지 않게 관리합니다. 원본은 보존 용도, 정리본은 꺼내 읽기 좋게 재구성한 결과물입니다.

- **매핑:** 하나의 대화 구간이 여러 노드에 걸쳐질 수 있고, 한 노드가 여러 구간에서 보장될 수 있습니다. 이 관계를 추적 가능하게 남깁니다.
- **갱신/누적:** 같은 주제를 다시 만나면 새로 만들지 않고 기존 내용을 보존하며 갱신하고, 언제 어떤 변경이 있었는지 버전 이력을 남깁니다.
- **저장 시 명시할 것:** 어떤 원본 구간이 어떤 노드에 반영됐는지, 어떤 변경이 있었는지(전/후), 제안이 수락됐는지 기각됐는지, 자동 반영이 아니라 사용자 선택으로 반영됐는지.

7.2 주요 API 엔드포인트

메서드	경로	설명
MCP	<code>/mcp</code> · <code>submit_conversation</code>	에이전트가 사용자 토큰으로 대화 원문 제출 (Streamable HTTP)
POST / GET	<code>/api/credentials</code>	MCP 연결 토큰 발급·목록·폐기
GET / POST	<code>/api/records</code>	보관된 대화 목록 조회 · 원문 등록 후 후보 생성
GET	<code>/api/proposals</code> , <code>/api/proposals/:id</code>	제안 목록·상세 (근거, 변경 전/후 포함)
POST	<code>/api/proposals/:id/decide</code>	수락/기각. 수락 시 노드 생성·갱신 또는 관계 저장
GET	<code>/api/nodes</code> , <code>/api/nodes/:id</code>	승인된 위키 노드 목록·상세
GET	<code>/api/graph</code>	생각 지도용 노드와 승인된 관계(edges)
GET	<code>/api/search/wiki-nodes</code> , <code>/api/search/chat-messages</code>	키워드 검색 (위키 노드 · 보관 대화)

8.1 MVP 성공 기준

- ✓ MCP로 제출한 대화가 사용자 계정의 유입 기록에 저장됩니다.
- ✓ 제출된 대화에서 최소 한 개 이상의 신규·갱신·연결 제안을 생성할 수 있습니다.
- ✓ 신규 제안을 수락하면 내 위키에 노드가 생성됩니다.
- ✓ 같은 주제의 후속 대화가 기존 위키 갱신 제안으로 이어집니다.
- ✓ 연결 제안을 수락하면 생각 지도에 두 위키의 관계선이 표시됩니다.
- ✓ 기각한 제안은 위키에 반영되지 않습니다.
- ✓ 새로고침 후에도 위키와 결정 상태가 유지됩니다.

8.2 추적 방법

- 위키 열람, 노드 클릭, 제안 노출/수락/기각, 관계 제시 노출/클릭 등 이벤트 설계
- 초기엔 과도한 이벤트보다 핵심 행동 위주로 추적

09 리스크 & 의존성

리스크	설명	완화 방안
제안 품질이 낮으면 스팸처럼 느껴질 수 있음	근거 부실/과도한 제안은 신뢰를 떨어뜨림	초반 보수적 제안, 근거 표기, 사용자 피드백 반영
자동 정리가 사용자의 통제감을 약화시킬 수 있음	위키가 사용자 의도와 다르게 정리될 수 있음	모든 반영을 수락/기각 승인 뒤에만 수행
대화가 너무 적으면 위키/제안이 빈약할 수 있음	초기 데이터 부족 시 가치 체감이 어려움	비어 있는 상태 UX 고려, 점진적 적립 경험 설계
비개발자 사용자에게 개념이 어렵게 느껴질 수 있음	신규/갱신/연결 같은 표현이 부담될 수 있음	친근한 표현, 짧은 설명, 점진적 노출

외부 의존성: Solar Pro 4 API(Upstage), MCP 클라이언트를 가진 에이전트(Hermes 등), PostgreSQL, 프론트/백엔드 호스팅.

10 일정 / 마일스톤

단계	기간	산출물 / 완료 기준
리서치 / 설계	예선 ~ 2026-09-11	핵심 사용자 유형, 위키 단위, 제안/관계 제시 방식, 데이터 모델 초안, PRD 0.1
MVP 개발	2026-09-12 ~ 09-16	MCP 수신·보관, 대화→위키 노드 생성, 기존 위키 대조 제안(신규·갱신·연결), 관계 제시 + 근거, 위키 조회/탐색/검색, 제안 수락/기각 기본 흐름
내부 테스트	2026-09-16	실제 대화 데이터로 위키 품질/제안 품질 확인, 흐름 점검
결선 제출 / 시연	2026-09-16 ~	핵심 흐름이 안정적으로 동작하고 비개발자 사용자도 쓸 수 있는 경험 확보. 제출 후 배포본 유지

11 부록

용어 정리

용어	뜻
ai-wiki 스킴	예선에서 만든 대표 스킴. 원본·정리본 분리, 병합/분리/연결 제안, 민감정보 경고, 갱신 이력 원칙 등 위키 운영 규칙을 담은 지침 문서(ai-wiki-SKILL.md)
Solar Pro 4	Upstage의 LLM. 이 서비스의 후보 생성과 관계 유도를 담당하는 유일한 모델
MCP	Model Context Protocol. 외부 에이전트가 우리 서비스의 <code>submit_conversation</code> 도구를 호출해 대화를 보내는 표준 통로
유입 기록 (Record)	보관된 대화 원문 한 건
세그먼트	원문을 근거 단위로 나눈 조각. 제안의 근거가 가리키는 대상
제안 (Proposal)	Solar가 만든 신규·갱신·연결 후보. 수락 전에는 위키에 영향을 주지 않음
위키 노드	승인된 정리본의 단위(주제/엔티티)
생각 지도	승인된 노드와 관계를 시각화한 그래프 홈 화면